

consider a matrix multiplication operation `tf.matmul(a,b)`, then it will create and add a matrix multiplication operation to the default graph to multiply matrices `a` and `b`. It will return a tensor `tf.Tensor` to represent the multiplication.

In most cases a default graph is sufficient to run a program, but in the case of multiple graphs, API `tf.estimator` is used to manage the default graph, and it uses different graphs for training and evaluation.

Session

An interactive session TensorFlow class is used in interactive contexts, such as a shell. This is convenient in interactive shells and IPython notebooks, as it is not required to pass an explicit session object to run an operation. The following example shows how a tensor constant `c` can be evaluated without an explicit call to a 'session run' operation. 'Method close' terminates an open interactive session.

```
sess = tf.InteractiveSession()
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
# We can just use 'c.eval()' without passing 'sess'
print(c.eval())
sess.close()
```

In the case of a regular session activation using a `with` statement, in non-interactive programs the tensor object is executed as follows:

```
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
with tf.Session():
    # We can also use 'c.eval()' here.
    print(c.eval())
```

Session method `as_default` returns a context manager that makes the session object a default session. When used by the `with` keyword `tf.Operation.run` or `tf.Tensor.eval` should be executed in this session.

```
c = tf.constant(599)
sess = tf.Session()
with sess.as_default():
    #assert tf.get_default_session() is sess
    print(c.eval())
```

Since the `as_default` context manager does not close the session upon exit from the context, it is mandatory to close the session explicitly on exiting the context. This can be avoided by invoking the session with `tf.Session()` directly within the 'with statement'. This class closes the session upon exiting from the context.

The default session is a property of the current thread. To attach the default session to a new thread, it is necessary to explicitly use `with sess.as_default()` in that thread. In the case of multiple graphs, if `sess.graph()` is different from the default graph (`tf.get_default_graph`), it is necessary to set `sess.graph.as_default()` for that graph.

list_devices

Session method `list_devices()` lists available devices in an activated session.

```
devices = sess.list_devices()
for d in devices:
    print(d.name)
```

This will display the full name of the device along with its type and the maximum amount of memory available. For example, a typical display of the above script may be `/job:localhost/replica:0/task:0/device:CPU:0`.

Slice

This operation is performed with `tf.slice()`, and it extracts a slice from a tensor object starting at the marked template location. The slice size is represented as a tensor shape, where `size[i]` is the number of elements of the `i`th dimension of the input that are to be sliced. The starting location *begin* of a slice is an offset in each dimension of the input. This starting location is zero-based, whereas the size is one-based. A value -1 of `size[i]` indicates that all remaining elements in dimension `i` are included in the slice and can be written as:

```
size[i] = input.dim_size(i) - begin[i]
```

For example:

```
t = tf.constant([[[[1, 1, 1], [2, 2, 2]],
                  [[3, 3, 3], [4, 4, 4]],
                  [[5, 5, 5], [6, 6, 6]]]])
init_op = tf.global_variables_initializer()
with tf.Session() as sess:
    sess.run(init_op)

    tf.slice(t, [1, 0, 0], [1, 1, 3])
    tf.slice(t, [1, 0, 0], [1, 2, 3])
    tf.slice(t, [1, 0, 0], [2, 1, 3])

    init_op = tf.global_variables_initializer()

    with tf.Session() as sess:
        sess.run(init_op)
        print( "Slice 1:",sess.run(tf.slice(t, [1, 0, 0], [1, 1, 3])), "\n")

    with tf.Session() as sess:
```